

Experiment 15

15.1.1 Car Dealership Sales System

Algorithm :

Step 1: Start

Step 2: Define base class Car

2.1 Method `__init__(self, brand, price, model, color)`

Assign `self.brand = brand`

Assign `self.price = price`

Assign `self.model = model`

Assign `self.color = color`

2.2 Method `display_details(self)`

Print `self.brand`

Print `self.price`

Print `self.model`

Print `self.color`

Step 3: Define derived class Car1 (inherits from Car)

3.1 Method `display_details(self)`

Call `super().display_details()`

(Calls `Car.display_details()` of parent class)

Step 4: Define derived class Car2 (inherits from Car)

4.1 Method `display_details(self)`

Call `super().display_details()`

(Calls `Car.display_details()` of parent class)

Step 5: Read input for car1

5.1 Read a line of input and split into parts

5.2 Assign `brand1 = first part`

5.3 Assign `price1 = float(second part)`

5.4 Assign `model1 = third part`

5.5 Assign `color1 = fourth part`

Step 6: Read input for car2

6.1 Read a line of input and split into parts

6.2 Assign `brand2 = first part`

6.3 Assign `price2 = float(second part)`

6.4 Assign `model2 = third part`

6.5 Assign `color2 = fourth part`

Step 7: Create objects

7.1 `car1 = Car1(brand1, price1, model1, color1)`

Calls `Car.__init__()` to initialize attributes

7.2 `car2 = Car2(brand2, price2, model2, color2)`

Calls `Car.__init__()` to initialize attributes

Step 8: Display details of car1

8.1 Call `car1.display_details()`

8.2 `Car1.display_details()` calls `super().display_details()`

8.3 `Car.display_details()` executes:

Print `brand1`

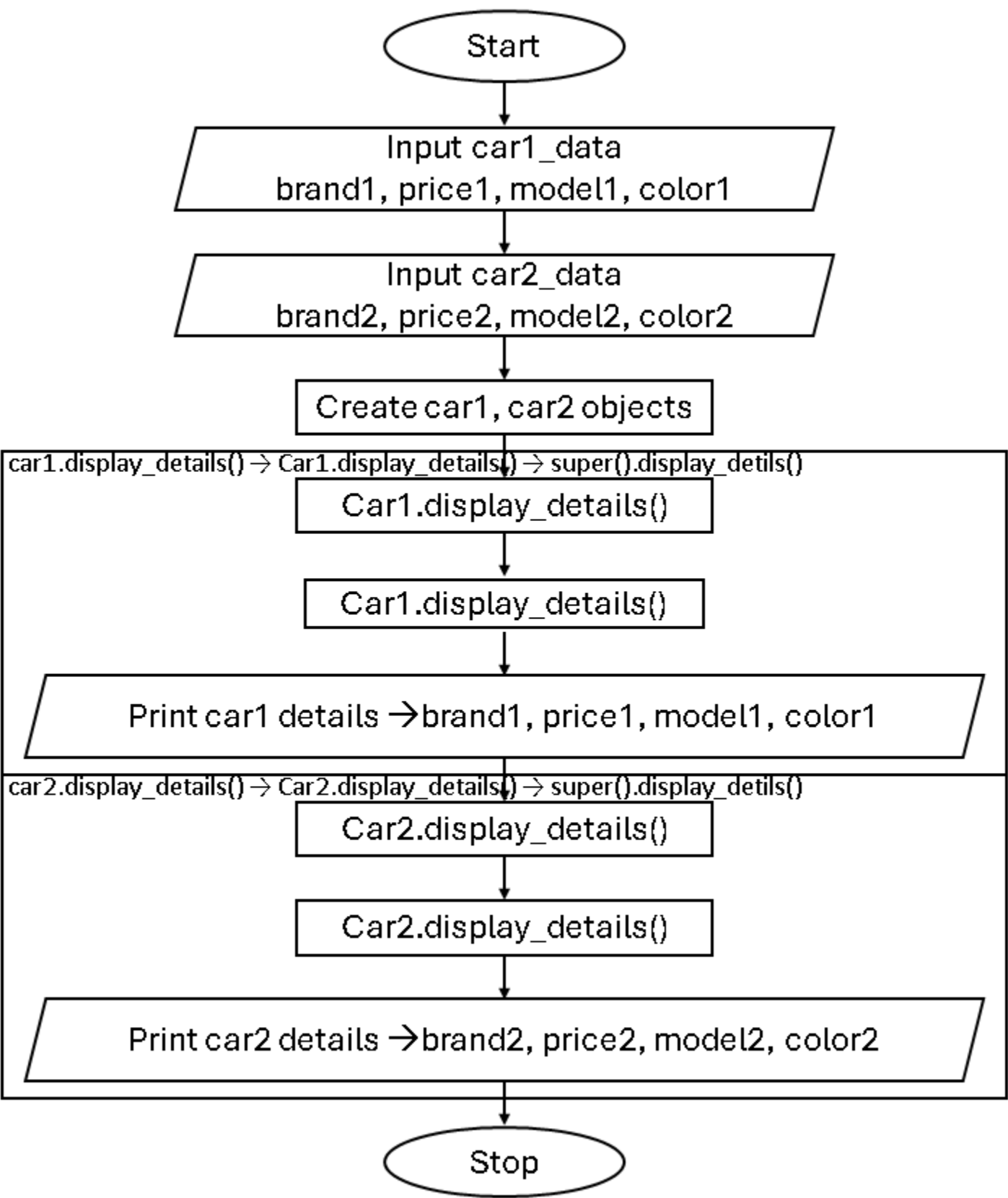
Print `price1`

Print `model1`

Print `color1`

- Step 9: Display details of car2
- 9.1 Call car2.display_details()
 - 9.2 Car2.display_details() calls super().display_details()
 - 9.3 Car.display_details() executes:
 - Print brand2
 - Print price2
 - Print model2
 - Print color2
- Step 10: Stop

Flowchart :



Code :

```
class Car:
    def __init__(self, brand, price, model,
color):
        self.brand = brand
        self.price = price
        self.model = model
        self.color = color
```

```
def display_details(self):
    print(self.brand)
    print(self.price)
    print(self.model)
    print(self.color)
```

```
class Car1(Car):
    def display_details(self):
        super().display_details()
```

```
class Car2(Car):
    def display_details(self):
        super().display_details()
        # Read input
        car1_data = input().split()
        brand1, price1, model1, color1 =
        car1_data[0], float(car1_data[1]),
        car1_data[2], car1_data[3]
```

```
car2_data = input().split()
brand2, price2, model2, color2 =
car2_data[0], float(car2_data[1]),
car2_data[2], car2_data[3]
```

15.1.1. Car Dealership Sales System

2012

Write a Python program to model a car dealership sales system using object-oriented programming. Create a base class `Car` and two derived classes `Car1` and `Car2` to represent different car types.

Class Structure

1. Base Class `Car`:

- Attributes: `brand`, `price`, `model`, `color`.
- Method: `display_details()` to print all car attributes.

2. Derived Classes `Car1` and `Car2`:

- Inherit from `Car` class.
- Override `display_details()` method to display car information in the specified format.

Input Format:

- The first line contains `Car1`'s `brand` (string), `price` (float), `model` (string), and `color` (string) separated by spaces.
- The second line contains `Car2`'s `brand` (string), `price` (float), `model` (string), and `color` (string) separated by spaces.

Output Format:

- First four lines: `Car1` details (`brand`, `price`, `model`, `color` - each on separate line)
- Next four lines: `Car2` details (`brand`, `price`, `model`, `color` - each on separate line)

Sample Test Cases

+

carDetail...

```
1 # Base class
2 class Car:
3     def __init__(self, brand, price, model, color):
4         self.brand = brand
5         self.price = price
6         self.model = model
7         self.color = color
8
9     def display_details(self):
10         print(self.brand)
11         print(self.price)
12         print(self.model)
13         print(self.color)
14
15 # Derived class Car1
16 class Car1(Car):
17     def display_details(self):
18         print(self.brand)
```

Average time

Maximum time

0.012 s

0.022 s

12.17 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Expected output

Toyota 25000.50 Camry White

Honda 22000.00 Accord Black

Toyota

25000.50

Camry

White

Actual output

Toyota 25000.50 Camry White

Honda 22000.00 Accord Black

Toyota

25000.50

Camry

White

Create objects

car1 = Car1(brand1, price1, model1, color1)

car2 = Car2(brand2, price2, model2, color2)

Display details

car1.display_details()

car2.display_details()

Execution :

Terminal

Test Cases

< Prev

Reset

Submit